```
.loop:
    xadd    rax,rdx
    loop    .loop
```

note: add <reg a>, <reg. b>
adds values of a & b & stores in register a

note: xadd <reg A>,<reg B>

① saves value @ reg A
② moves val @ reg B (exchange) → reg A & & stored → B
③ adds same value to dest (reg A)

d s

xadd rax, rdx

temp = rax value
rax = rdx value
rdx := temp
(values)
rax += temp

loop .loop

→ decrements
→ RCX == 0?
Y → stop
N → goto .loop

(loop reg) rcx register

$d_1 = s_0 + d_0$
$s_1 = d_0$
↓ loop ...
$d_2 = s_1 + d_1$
$s_2 = d_1$
↓ ...
$d_n = s_{n-1} + d_{n-1}$
$s_n = d_{n-1}$

$d_0 = 1, s_0 = 1$

$d_1 = 1 + 1 = 2$
$s_1 = 1$
$d_2 = 2 + 1 = 3$
$s_2 = 2$

... fib???

[0x01]

let rax = c
rdx = d, ⇒ c_n, d_n

$c_n = c_{n-1} + d_{n-1}$
$d_n = c_{n-1}$

5 fib reg.

```
.loop:
    xadd    rax, rdx
    loop    .loop
```

ex	rcx := f	rax := 1	rdx := 0	
(1,1)	1	e	1	
(2,1)	2	d	1	
(3,2)	3	c	2	
(5,3)	5	b	3	
(8,5)	8	a	5	
(d,8)	d	9	8	
(x15,xd)	15	8	d	
(x22,x15)	22	7	15	Ethex
(x37,x22) ⇐	37	6	22	
(x59,x37)	59	5	37	
(x90,x59)	90	4	59	
(xe9,x90)	e9	3	90	
(x179,xe9)	179	2	e9	
(x262,x179)	262	1	179	
(x3db,x262)	3db	0	262	

[0x01]